I-JEPA 从零实现：从论文方法到可检验的训练系统

对象：刚开始研究、已具备 Python/PyTorch 基础的研究生

版本：1.0（2026-10-09）

主要依据：Assran et al., Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture, arXiv:2301.08243v3（本目录上一层的论文 PDF）。

结构参考：CS336 Assignment 1 的“背景 → 小题 → 接口 → 测试 → 综合训练”组织方式；这里的题目和测试是针对 I-JEPA 重新设计的。

1. 作业目标与边界

完成后，你应能独立回答：I-JEPA 预测什么？为什么预测表征而非像素？为什么目标分支使用完整图像和 EMA？掩码如何让任务既有语义又非平凡？最后，你应能从零训练一个小型 I-JEPA，并给出自我监督训练之外的评价证据。

你将实现：（1）patch 与索引；（2）多块掩码；（3）ViT 编码器；（4）带位置条件的预测器；（5）表征损失与 EMA；（6）训练、断点恢复、冻结特征评估；（7）消融。测试只验证局部正确性，不会替代训练实验。

“从零实现”指自己编写模型和训练链条；可使用 PyTorch 张量、自定义 nn.Module、基础优化器与数据加载器。不要调用现成的 I-JEPA、ViT 或 masked-image-modeling 实现来完成核心题。若你已有 CS336 中亲自实现的注意力/MLP，可复用自己的代码，并解释它们与这里的非因果图像注意力有什么不同。

两档复现：必做是机制复现（小模型与小数据）；论文级 ImageNet 指标只作扩展。论文附录 A 使用 batch size 2048、长期预训练和大型 ViT；本练习的 CPU 测试或小数据实验不会出现论文的 Top-1 数字。

2. 方法从哪里来

2.1 三条路线及其矛盾

论文第 1–3 节先比较三类自监督学习：联合嵌入方法使同一图像的两个增强视图接近，但增强方式会写入任务偏置；生成式遮挡建模从可见部分重建像素/离散 token，必须关心很多预测任务不需要的低层细节；JEPA 在表征空间预测缺失内容，力图让预测聚焦于更可预测、更抽象的信息。这个推理是研究动机，不是“表征空间一定产生语义”的数学保证。

为什么还需要谨慎？若所有输入都映射为同一个向量，预测误差也可能很低，这就是表征坍塌。I-JEPA 使用不对称的在线上下文编码器、预测器，以及不反传梯度的 EMA 目标编码器来缓解它。仅凭训练损失不能排除坍塌；要观察不同样本特征方差与下游评价。

从“重建缺失信息”走到 JEPA，可以把困难拆开看。像素目标包含纹理、微小位置偏差和不可由上下文推断的细节；若这些细节占据大量损失，模型会花容量拟合它们。把监督目标改成一个随训练更新的编码器输出后，模型有机会忽略这些细节。但编码器也可能选择把一切都忽略，因此需要研究表示是否坍塌。I-JEPA 的实验主张是：合适的多块掩码和不对称的 EMA 结构，使这种表征预测在图像上可行。论文第 9 节的像素目标对比、附录 C 的目标取块位置对比支持这一设计；它们不等于严格证明。把“动机—设计—实验支持—仍未证明”分开写，是本作业的方法理解要求。

思考题 A（书面）：分别用 2–3 句话比较像素重建、两视图不变性和 I-JEPA 的监督信号。举一个“增强操作会破坏下游信息”的例子，并说明为什么此例仍不足以证明 I-JEPA 必然优于另两者。

2.2 用一句话描述 I-JEPA

输入一张图，目标编码器从完整图像生成所有 patch 表征；在线编码器只看一个较大的、扣除了目标区域的上下文；预测器在目标位置 token 的条件下分别预测多个较大目标块的表征；优化预测误差，并用在线编码器的 EMA 更新目标编码器。对应论文图 3、第 3 节和附录 A.1。

设图像切成 N=H_g×W_g 个 patch，在线编码器为 f_θ，目标编码器为 f_θ□，预测器为 g_φ。目标块 B_i 可彼此重叠，上下文索引集 B_x 必须与所有 B_i 不相交。目标表征 S_y=f_θ□(image) 先在完整图像上求得，再按 B_i 取出；上下文表征 S_x=f_θ(image, B_x) 只处理可见 patch。每块预测 P_i=g_φ(S_x, B_x, B_i)。

本作业将论文第 3 节的目标明确为

L = (1/B) Σ_b [(1/M) Σ_i Σ_(j∈B_i) ||P[b,i,j] - S_y[b,j]||²□]。

这里对 batch 求平均、对目标块求平均，块内 patch 求和、特征维求和。论文公式没有把块大小或特征维再除掉；若使用通用 MSELoss(reduction='mean')，量纲和权重会改变。重叠的目标 patch 在其各自块中各计一次。

2. 3 一次前向传播的地图

阶段	输入与输出	是否由反向传播更新
目标编码器	完整图像 [B,C,H,W] → 所有目标 token [B,N,D]，然后按目标索引取块	否；只用 EMA
上下文编码器	图像与可见索引 [K] → [B,K,D]；隐藏 token 不进入 Transformer	是
预测器（每个目标块一次）	上下文 token、上下文位置、目标位置 [T_i] → [B,T_i,D]	是
损失	每块预测与目标表征对齐，特征/patch 求和，目标块/batch 平均	梯度仅回到在线分支

例如一张 2×3 patch 网格按行优先编号为 0,1,2 / 3,4,5。如果两个目标分别是 {1,2} 和 {2,5}，编号 2 可以在两个目标中出现；上下文若原为整张网格，扣除两个目标的并集后只剩 {0,3,4}。目标编码器仍处理编号 0…5 的全部 patch。这幅小图可以作为调试每个索引函数的共同样例。

损失也可以先用标量手算：令 batch 只有一张图、两块目标的预测误差分别为 [1] 和 [2,3]，那么平方 L2 的块内和分别为 1 与 4+9=13，最后损失为 (1+13)/2=7，不是 (1+4+9)/3。若目标 2 在两块中出现，它在两块中分别产生误差项；这是两个预测任务，不是索引去重问题。

思考题 B（书面）：画一幅 4×4 patch 网格，画两个可重叠目标块和一个上下文矩形。写出 B_x 与两个 B_i。解释为什么“目标输出后取块”与“目标输入先遮挡”不是同一个学习问题。

3. 约定、仓库与运行

图像张量 [B,C,H,W]，patch 网格 [H_g,W_g]，其中 H_g=H/P, W_g=W/P 且 P 整除高宽。patch 序号使用行优先 j=row×W_g+col。token 张量 [B,N,D]。传入索引为一维 LongTensor[K]；测试也覆盖一些二维 [B,K] 的收集操作。目标块索引列表每项一维，升序、无重复；不同目标块之间可重复。调试时先用方形输入，代码不要把行数和列数偷换。

在 ijepa_student/ 下自由组织自己的代码。tests/adapters.py 列出评分接口；将它们接到自己的函数或模型构造器，不要把实质算法写进 adapter。按顺序运行 python -m pytest -q tests/test_masking.py 等。最初因 adapter 抛出 NotImplementedError 而失败是设计使然。

4. 第一阶段：patch、位置和多块掩码

4. 1 Patch 和 token 收集

问题 1（patches）：实现 patchify(images, patch_size)，返回 [B,N,C·P²]，patch 排列按行优先，patch 内也按 C,H,W 的顺序展平。实现 gather_tokens(tokens, indices)，支持 [K] 和 [B,K] 索引；输出 [B,K,D]，必须保留输入索引顺序。

交付：源码、测试通过、手算一个 1×1×4×6 图像在 P=2 时各 patch 的前后顺序。测试：test_masking.py。

4.2 先构造确定性几何，再随机采样

问题 2（`mask_geometry`）：build_context_indices(grid_size, context_box, targets) 的 context_box=(top,left,height,width)，先列出该矩形中的行优先 patch，再删除所有目标索引的并集。验证坐标和索引范围；保留上下文的升序。

问题 3（`sample_masks`）：实现 sample_masks(grid_size, generator, num_targets, target_scale, context_scale, target_aspect)。scale 是相对于整张 `patch` 网格面积的比例；目标块为长宽比在给定范围内的矩形；上下文先采样一个近正方形的大矩形，再扣掉全部目标块。返回非空的一维上下文索引与 num_targets 个非空一维目标索引，所有集合在单个图像内可复现。可以自主决定取整和重采样细节，写在报告中；测试不规定特定随机数流或具体坐标。

论文默认 M=4、目标 scale (0.15,0.2)、目标 aspect (0.75,1.5)、上下文 scale (0.85,1.0)、上下文 aspect 1。论文允许目标块相互重叠；上下文与任何目标不能重叠。附录 A.1 说明每张图独立采样，批处理时让同一设备上掩码长度一致以利于堆叠。练习先做单图采样，之后在数据 collator 中按长度协调批次；不要为了批处理让整批图共享同一个位置掩码。

交付：可视化至少 8 组掩码（可以文本网格），报告目标面积、上下文剩余面积、重叠情况；test_masking.py 通过。低资源提示：先在 16×16 网格调试，再考虑大图；随机拒绝采样时设置有限次数，并提供明确失败错误。

阶段验收：不同随机种子通常产生不同掩码；同一生成器初始状态必然重现同一组掩码；每个目标是一块完整矩形；上下文与目标并集的交集为空。请用一个非方形网格再手工检查行优先坐标，避免代码只适用于正方形。

思考题 `C`：为什么要较大的目标块？为什么上下文要包含足够多且分布较广的 patch？论文表 6 和附录表 8–10 给出了哪些经验支持？只描述论文观察，不把观察当作普遍定理。

5. 第二阶段：从零构造编码器

5.1 不带因果掩码的 ViT

问题 4（`attention`）：实现缩放点积注意力 softmax(QK□/√d) V，可有前导 batch/head 维度，不加因果掩码。再实现多头自注意力、前馈网络、残差与归一化构成的 Transformer block。论文使用 ViT；具体 block 的内部数值选择（如前归一化）由你记录，但不要引入会破坏测试确定性的随机失活。

问题 5（`encoder`）：build_encoder(image_size, patch_size, in_channels, embed_dim, depth, num_heads, mlp_ratio) 返回 nn.Module。forward(images, indices=None)：先将图像转为 patch embedding 并加入绝对位置嵌入（不同位置的初始向量应可区分）；若有 indices，在 `Transformer` 前仅保留这些 token；若无索引，处理整张图的所有 token。输出 [B,K,D] 或 [B,N,D]。预训练不使用 [cls] token。

不能把隐藏 patch 填零后仍让它们参与自注意力；这会改变计算图，也可能造成信息泄漏。test_model.py 用两张仅在不可见 patch 不同的图像检查上下文输出应相同。由于目标编码器处理整图，其输出可以随这些变化而变化。

交付：test_model.py 中注意力和编码器测试通过；列出每个阶段张量形状与一个关于可见 token 数的断言。思考题 `D`：图像自注意力为什么一般不使用语言模型的因果掩码？删除 token 与填零 token 的计算和信息流有何区别？

阶段验收：给同一张图选择 5 个可见 patch，Transformer 的 token 轴应确实为 5。再把所有隐藏 patch 的像素改变为极大数值；可见输出在浮点容差内不应变化。只检查最终输出形状不够，因为“处理完整图后再 gather”也会得到 [B,5,D]，但它泄漏了隐藏图像。

5.2 目标分支

问题 6（`target_encoder`）：从在线编码器深拷贝出目标编码器，初始化参数相同、关闭梯度。每一步都先对完整图像计算目标 token，再用目标索引取块。目标编码器不会被 optimizer 管理。

交付：展示一次反向传播后在线编码器与目标编码器参数的 .grad 状态，并解释结果。论文附录 A.1 与表 11 提示：若先在目标输入上遮挡，目标 token 缺少整个图像上下文，学到的目标发生变化。

6. 第三阶段：位置条件预测器与损失

问题 7（predictor）：build_predictor(context_dim, predictor_dim, depth, num_heads, target_dim, num_patches) 返回模块，forward(context_tokens, context_indices, target_indices) 输出 [B,T,target_dim]。把上下文 token 映到较窄维度；给每个待预测位置一个共享可学习 mask 向量加该位置的嵌入；上下文 token 也保留其原始位置；拼接后由非因果 Transformer 处理；只取目标 token 并映回 target_dim。每个目标块单独调用预测器，共用参数。测试目标顺序置换后输出要对应置换。

论文附录 A.1：预测器是较窄的 ViT，维度 384；ViT-B 的预测器深度 6，更大的部分模型使用 12 或 16。小型练习可以缩至 predictor_dim=32、depth=2，但计算结构保持一致。

问题 8（loss）：实现第 2.2 节的 jepa_loss(prediction_blocks, full_target, target_indices)。检查每块的 token 数、维度和设备；目标分支应 detach 或在无梯度上下文运行。测试包含不等大小目标块与重叠索引，确保没有误用全局平均。

交付：test_model.py、test_objective.py 通过；手算一个两块各 1 和 2 个 token 的损失。思考题 E：预测器为什么需要目标位置？若移除它，哪类目标会变得不可区分？为什么同一目标 patch 在两块中出现可以计两次？

阶段验收：保持上下文不变，将目标位置索引重排，预测输出应随之重排；将目标位置换成另一处，输出一般应改变。检查一个块的输出只对应这个块的索引，不能在多块循环中重用上一个块的位置。目标编码器输出和索引提取可在 no_grad 中完成。

7. 第四阶段：EMA 与完整训练步

问题 9（ema）：目标参数从在线参数复制初始化；每次 optimizer 更新之后执行 $\theta_{\square} \leftarrow m\,\theta_{\square} + (1-m)\,\theta$。实现 ema_update 与 momentum_at(step,total_steps,start)，后者在 step=0 为 start，在 step=total_steps 为 1，之间线性变化。训练时目标分支不接收梯度；不得把目标参数加入 optimizer。

自检例子：某个标量目标参数更新前为 2，在线参数在 optimizer 步后为 6，若 m=0.75，目标更新后应为 3。若得到 2.5，你可能用了 optimizer 步之前的在线参数；若得到 6，你可能直接复制了在线分支。

问题 10（pretrain_step）：实现一次图像 batch 的完整训练步：目标分支整图前向和索引提取；上下文分支仅可见 token；按目标块分别预测；计算损失；清梯度、反传、更新在线编码器和预测器；随后 EMA。返回 Python float。逐步检查所有输出维度。

论文附录 A.1 用 AdamW，学习率前 15 epoch 从 1e-4 预热到 1e-3，之后余弦下降至 1e-6；weight decay 从 0.04 线性到 0.4；momentum 从 0.996 线性到 1.0。这些是论文训练配置，不是 CPU 小模型必须照搬的数值。小模型先找能稳定训练的配置，并完整记录差异。

问题 11（checkpoint）：保存并恢复在线编码器、目标编码器、预测器、optimizer 和完成的 step。重新载入后，同一输入与掩码的损失应相同。若要实现严格续训，还需保存随机数生成器状态、数据迭代位置和调度器状态；报告中说明是否已实现。

交付：test_training.py 通过；一次小数据训练的损失曲线、梯度与特征方差日志、断点恢复实验。低资源提示：先用 64×64 图、P=8、D=64、浅层 ViT，优先验证训练链条，之后扩大。

阶段验收：一次训练步应只使在线编码器、预测器从优化器得到更新。若 m=1，目标参数本步保持原值；若 m=0，目标参数本步等于优化器更新后的在线参数。逐项记录参数范数可定位更新次序错误。断点恢复后先在相同输入与相同掩码上比较输出，再比较继续训练；后者还取决于 RNG 与数据迭代状态。

8. 第五阶段：实验与评价

8.1 最小训练实验

使用具有重复结构的无标签图像（如合成几何图、自己有权使用的图像集）训练。训练/验证按原始图像划分；同一图像的不同掩码不要分散在训练与验证集合。每隔固定步记录：训练与验证预测损失、目标/在线特征在样本维的标准差、可见 token 比例、每步时间与显存（若有 GPU）。绘制掩码图以排除索引错误。

诊断顺序：先让单 batch 的损失下降；再在完整小训练集上运行；最后看冻结特征是否有用。若损失下降、特征方差趋近零，应怀疑坍塌。若上下文看到目标位置，应怀疑泄漏。训练损失本身不是表征质量指标。

8.2 冻结特征线性探针

论文附录 A.1 评价时使用目标编码器，并平均池化所有 patch 的输出。本作业 encode_for_probe(images,target_encoder) 就采用这一定义，返回 [B,D]。用一个仅在标注训练集上训练的线性分类器评价冻结特征；报告随机初始化编码器、预训练编码器的同协议结果及样本数。若无合适标签，可做近邻检索或可视化，但明确写明这并不等同于线性探针。

8.3 至少两项消融

建议选择：（a）表征预测 vs 像素预测；（b）目标输出取块 vs 目标输入遮挡；（c）多块 vs 单块；（d）大 vs 小目标；（e）有 vs 无目标位置嵌入。一次只改一个因素，固定数据划分、训练步数和算力。论文第 9 节与附录 C 对（a）–（d）提供了方向性证据，但你的低资源结果可能不同；重点解释机制和限制。

最终交付清单：源码、可复现环境说明、全部测试结果、训练命令和配置、损失/方差曲线、至少一项冻结特征评价、两项消融、1–2 页方法说明（含信息流图、与论文配置差异、失败或未证实的部分）。

9. 常见错误速查

现象	先检查
损失近乎零但探针很差	特征方差、目标分支是否坍塌、上下文是否看见目标
单 batch 也不下降	损失归约、梯度是否进入在线分支和预测器、目标 token 索引
掩码测试失败	面积比例相对整图、行优先索引、扣除全部目标并集、随机种子
换掉隐藏像素后上下文输出变化	是否在 Transformer 前移除隐藏 token、patch 是否对齐
EMA 测试失败	初始化相同、目标分支冻结、先 optimizer 后 EMA、是否误更新预测器
探针维度不对	预训练无 CLS；用目标编码器完整图输出的平均池化

10. 论文定位与复现声明模板

阅读建议：先看图 2–4 与第 2–3 节，建立信息流；再看第 9 节及附录 C，理解掩码和表征目标的证据；最后看附录 A.1 的架构、训练和评估细节。表 1–5 是结果而非作业通过标准。

报告中请写清楚：“我们复现了 I-JEPA 的 [哪些机制]；使用 [数据/模型/训练步数]，与论文的 [差异]；得到 [局部测试/训练/下游评价] 证据；未验证 [论文级比较]。”不要把玩具数据 loss 曲线描述为论文结果复现。

参考：Assran et al., Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture, arXiv:2301.08243v3，重点第 2–3、9 节及附录 A、C。组织方式参考 CS336 Assignment 1。